



SKYLINE WEATHER

A Production-Quality Full-Stack Weather Application

React (Vite) + Node.js / Express + OpenWeatherMap API

Class Presentation

What the App Does

Core capabilities delivered end to end, from search to display



Search by City

Look up current conditions for any city name



Use My Location

Browser Geolocation API pulls weather for wherever you are



Full Conditions

Temp, feels-like, humidity, wind, pressure, visibility, sunrise/sunset



5-Day Forecast

Forward-looking outlook alongside current conditions



Favorites & History

Star cities and revisit recent searches, saved in Local Storage

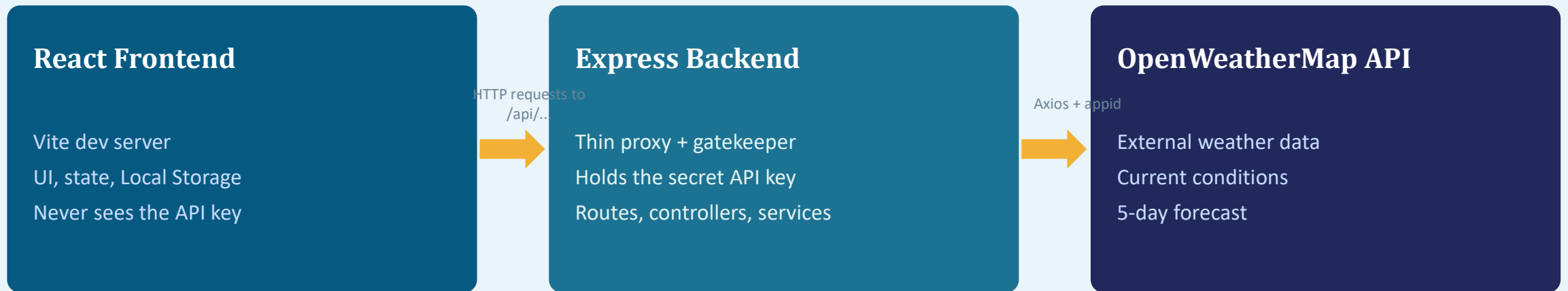


Dark / Light Mode

Instant theme toggle with dynamic weather-based gradients

System Architecture

A classic client-server design keeps the API key private



The frontend never talks to OpenWeatherMap directly — the backend calls it server-side, keeping the API key out of the browser and leaving room for caching or rate-limiting later.

Project Structure

Two independent apps, cleanly separated by responsibility



client/ — React (Vite)

- api/ — shared Axios instance
- components/ — SearchBar, WeatherCard, Forecast, Loader, ThemeToggle, FavoriteCities, SearchHistory
- context/ — ThemeContext (dark/light)
- hooks/ — useWeather (data fetching)
- utils/ — formatting + theme helpers
- App.jsx, App.css, index.css, main.jsx

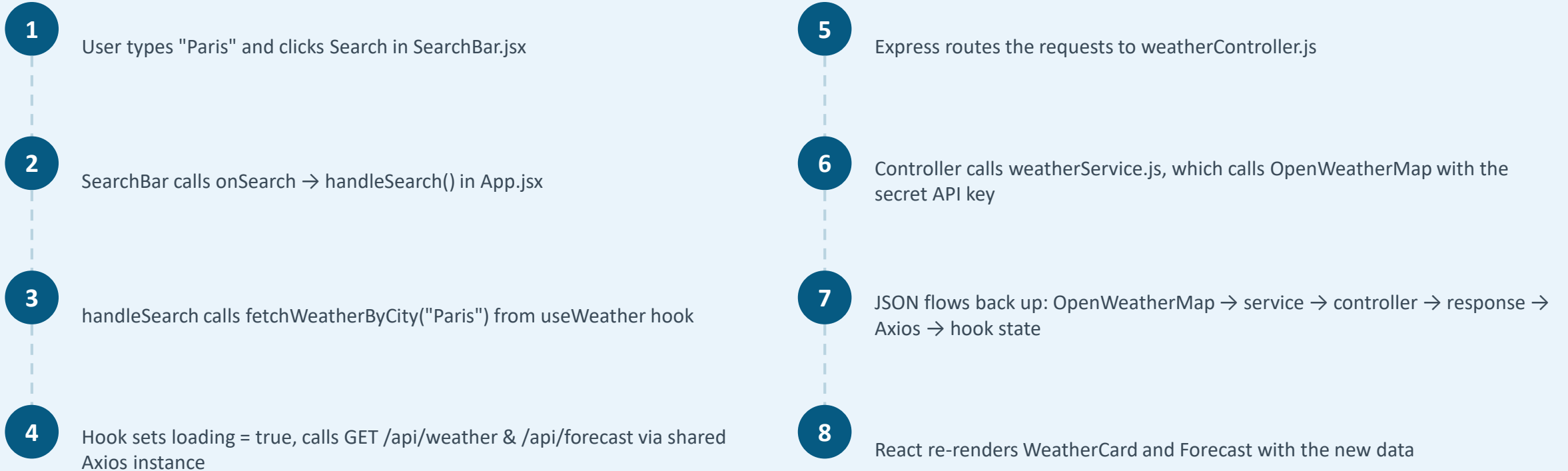


server/ — Node.js + Express

- config/ — env-based configuration
- controllers/ — request handlers
- middleware/ — centralized error handler
- routes/ — API route definitions
- services/ — OpenWeatherMap API calls
- index.js — Express app entry point

Data Flow — Searching "Paris"

One user action, eight steps, top to bottom



State Management

No Redux needed — built-in React tools cover it all

useState

Local component state for inputs and dropdown visibility

useWeather (custom hook)

Server data, loading, and error state — reusable and testable

React Context

ThemeContext holds the one truly global piece of state (dark/light mode)

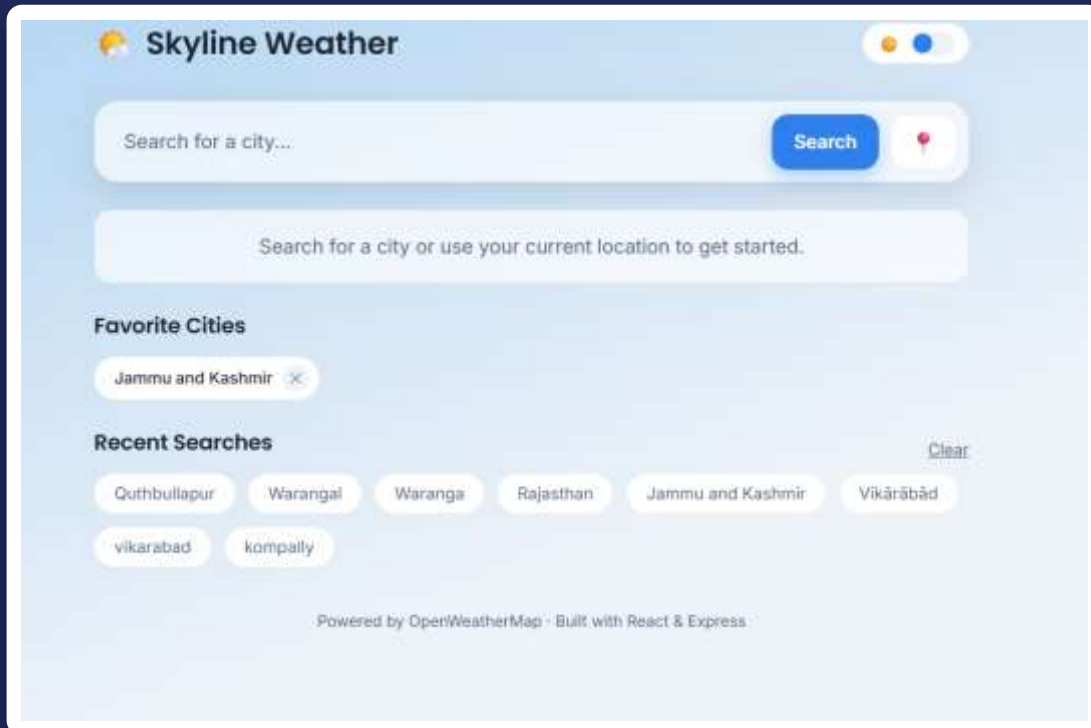
Local Storage

Persists history, favorites, and theme; read on mount, written on change

Data flows down via props; events flow up via callback props (e.g. onSearch, onToggleFavorite) — one-directional and predictable.

The App in Action

Empty state with favorites & history, plus a full weather result with 5-day forecast



Home — search, favorites & recent history



Result view — current conditions & 5-day forecast

Setup, Testing & Deployment

From local dev to a live, deployed app



Environment Variables

server/.env

OPENWEATHER_API_KEY, PORT,
OPENWEATHER_BASE_URL

client/.env

VITE_API_BASE_URL — the VITE_ prefix exposes
it to frontend code



Testing Checklist

- Valid city search returns weather + forecast
- Invalid city shows a friendly error, not a crash
- Geolocation loads weather for current position
- Favorites persist after a page refresh
- Recent searches populate the history list
- Dark mode toggle persists on reload
- Layout adapts down to mobile widths



Deployment

Backend

Render / Railway / Fly.io / Heroku — build: npm
install, start: npm start

Frontend

Vercel / Netlify / GitHub Pages — build: npm run
build, output: dist

CORS

Restrict cors() to the deployed frontend origin in
production

Future Improvements

Where the project could go next



Automated Tests

Jest + React Testing Library on the frontend,
Supertest for backend routes



Hourly Forecast Charts

Visualize hour-by-hour trends with Chart.js or
Recharts



Air Quality Index

Pull AQI data via OpenWeatherMap's Air Pollution API



City Autocomplete

Use the Geocoding API to suggest cities while typing



Offline Support (PWA)

Add a service worker for installability and offline use



Response Caching

Redis-based caching to cut repeat OpenWeatherMap
calls



Thank You

Skyline Weather — React, Express, and OpenWeatherMap working together

Questions?